

Communication Models

Choosing the shape of an interaction, and writing down the contract

Prof. Anis Koubaa

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 3, Lecture A



What this lecture does

Roadmap

Last week everything was a topic. Today: the three shapes an interaction can have, how to tell which one you need, and how to write the contract down so a compiler — not a robot at 3 a.m. — catches your mistake.

- 1 Three shapes of interaction
- 2 Writing the contract
- 3 Services in detail
- 4 Parameters
- 5 Actions

Three shapes of interaction

The question to ask before you write anything

“Is this a **stream**, a **question**, or a **job**?”

Stream
never ends
nobody waits

topic

Question
answered at once
caller waits

service

Job
takes time
may be cancelled

action

Getting this wrong does not produce an error message. It produces a robot that works in the lab and hangs on demo day. The three failure modes are on the next four slides.

Deciding, in practice

What you are doing	Shape	Why
Publishing LiDAR scans	topic	Continuous. Nobody waits for a scan.
Sending wheel velocities	topic	Continuous. The next one supersedes it.
“Reset the odometry to zero”	service	One question, answered immediately
“Is the battery healthy?”	service	A value you need <i>now</i> , on demand
“Drive to the kitchen”	action	Forty seconds. You want progress. You may cancel.
“Follow this trajectory”	action	Same: long, cancellable, worth reporting on

The two-second test

If the caller would be unhappy blocking until it finishes, it is not a service. If nobody is waiting for an answer at all, it is a topic.

Failure 1 — a topic where you needed an answer

You publish “reset the odometry” on a topic and carry on.

Did anyone receive it?	You do not know.
Did it succeed?	You do not know.
Was anyone even listening?	You do not know.

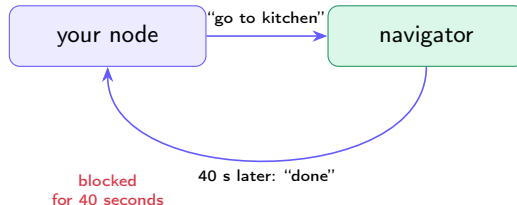
A topic is **fire and forget**. That is a feature for a scan arriving ten times a second — the next one repairs any loss. It is a defect for a command that happens once.

In the field

This is why `/cmd_vel` is still a topic and “reset odometry” is not. A dropped velocity is corrected 50 ms later. A dropped reset is silently never done.

Failure 2 — a service for something slow

“Navigate to the kitchen” as a service. It is one request and one reply, so it fits.



For forty seconds your node answers nothing, reports nothing, and cannot be told to stop. If an obstacle appears, or the user changes their mind, there is no mechanism to say so.

No feedback. No cancellation. No visibility. That is what actions exist to fix.

Failure 3 — the deadlock you will write at least once

Calling a service **synchronously, from inside a callback**, is the classic ROS 2 hang:

What you wrote	What happens
A subscriber callback that calls a service and waits for the reply	The callback blocks. <code>spin</code> is inside that callback, so it cannot process the reply. The reply never arrives.

Why

`spin` is single-threaded by default. It runs **one** callback at a time. A callback that waits for something `spin` must deliver is waiting for itself.

The fix is `call_async` and a future. You will cause this on purpose in session B — it is much easier to remember once you have watched your own node hang.

Writing the contract

A message definition, field by field

A .msg file is a list of *type name* pairs. That is all it is.

ee414_w03_interfaces/msg/WheelState.msg

```
std_msgs/Header  header      <- stamp + frame_id
float64          left_rad_s   <- left wheel angular velocity
float64          right_rad_s  <- right wheel angular velocity
bool             estopped
```

Primitives	bool int8 int32 int64 float32 float64 string
Arrays	float64[] unbounded, float64[16] fixed, float64[<=8] bounded
Nested	any other message type, e.g. geometry_msgs/Pose

These compile into Python and C++ classes. A misspelt field is a build error.

When to define a custom message — and when not to

Define one when

- The data genuinely has no standard type
- Fields must travel together, atomically, with one timestamp
- It is your own robot's internal state

Do *not* when

- A standard type already fits — use it
- You only want to rename fields
- You are avoiding reading the standard definition

In the field

Every custom type is a wall. Publish your own `MyLaserScan` and `RViz2` cannot display it, `slam_toolbox` cannot consume it, and nothing in the ecosystem knows what it is. Standard types are how your node talks to software you did not write — which is the entire value proposition of Weeks 9 through 11.

Services and actions: the same idea with separators

srv/ResetOdom.srv

```
bool zero_heading
---
bool success
string message
```

Request above the line, response below.

action/DriveDistance.action

```
float64 metres
---
bool reached
float64 travelled
---
float64 percent_done
```

Goal, result, feedback.

Three sections is what an action *is*: something you ask for, something you eventually get, and something you hear about while you wait.

Interfaces live in their own package

```
ee414_ws/src/  
  ee414_w03_interfaces/    <- ament_cmake. ONLY .msg/.srv/.action files.  
  ee414_w03_nodes/        <- ament_python. Your Python nodes.
```

Two reasons, both of which matter before Week 12:

- 1 A Python package **cannot generate interfaces** — generation is a CMake job, so definitions need an `ament_cmake` package.
- 2 Anyone who wants to *talk* to your robot needs the definitions but not your code. They install one small package, not your whole project.

Under the hood

This is why `geometry_msgs` and `sensor_msgs` are packages of their own. Same pattern, larger scale.

Reading a definition you did not write

```
$ ros2 interface show sensor_msgs/msg/LaserScan
std_msgs/Header header
float32 angle_min
float32 angle_max
float32 angle_increment
float32 range_min
float32 range_max
float32[] ranges
```

Do this before writing a single line against an unfamiliar topic. It is faster than searching, it is never out of date, and it is exactly the field list your code must use.

`ros2 interface list` shows everything available; `ros2 topic type /scan` tells you which one a topic carries.

Services in detail

The client is asynchronous. Always.

```
self.cli = self.create_client(ResetOdom, 'reset_odom')
req = ResetOdom.Request()
req.zero_heading = True

future = self.cli.call_async(req)
future.add_done_callback(self.on_reply)

def on_reply(self, future):
    self.get_logger().info(f'success: {future.result().success}')
```

`call_async` returns immediately with a **future** — a promise of a result later. You attach a callback and return control to `spin`, which is then free to deliver the reply.

This shape avoids failure 3. Get used to it now; it returns for actions.

Calling a service without writing a client

```
$ ros2 service list
/reset_odom
$ ros2 service type /reset_odom
ee414_w03_interfaces/srv/ResetOdom
$ ros2 service call /reset_odom \
    ee414_w03_interfaces/srv/ResetOdom "{zero_heading: true}"
response: success=True, message='odometry zeroed'
```

You can exercise a server completely from the command line before its client exists — which means you can test the two halves separately, and know which one is broken.

Same idea as `ros2 topic pub` last week. **Build one half, test it from the terminal, then build the other.**

Parameters

Never hard-code a number you will want to change

```
self.declare_parameter('max_speed', 0.22)
self.declare_parameter('frame_id', 'base_link')

speed = self.get_parameter('max_speed').value
```

A parameter is a named value the node reads at run time. Declared once, with a default, then read wherever it is needed.

Why this is not over-engineering

Every tuning constant in this course — controller gains in Week 5, the obstacle threshold in Week 6, costmap sizes in Week 11 — is a parameter. Tuning means changing a number and running again. If that requires an edit and a rebuild, you will try five values. If it is a parameter, you will try fifty.

Setting them from outside the code

```
$ ros2 run pkg node --ros-args -p max_speed:=0.15

$ ros2 param list /my_node
$ ros2 param get /my_node max_speed
$ ros2 param set /my_node max_speed 0.30      <- while it runs
```

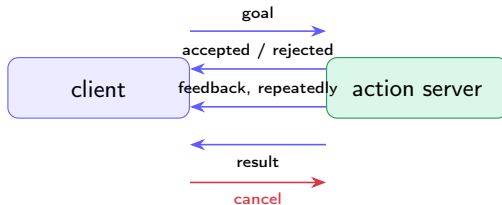
And in a launch file, which is where they belong for anything reproducible:

```
Node(package='pkg', executable='node',
      parameters=[{'max_speed': 0.15}])
```

Setting a gain live, while the robot drives, is how tuning is actually done. You will do exactly this in Week 5.

Actions

What an action adds



Five exchanges instead of two. The server may **reject** a goal outright, reports **progress** while it works, and the client may **cancel** at any point.

Underneath, an action is built from topics and services. You never see that — but it explains why an action has more moving parts than either.

Sending a goal from the command line

```
$ ros2 action list
/fibonacci
$ ros2 action send_goal /fibonacci \
    action_tutorials_interfaces/action/Fibonacci "{order: 8}" --feedback
Goal accepted with ID: 4f2c...
Feedback: partial_sequence: [0, 1, 1, 2]
Result: sequence: [0, 1, 1, 2, 3, 5, 8, 13]
```

--feedback is the flag that shows what an action gives you that a service cannot: **you can watch it work.**

In the field

In Week 11 you send a goal to Nav2's `navigate_to_pose` action exactly this way. Feedback is the distance remaining; cancel is what a safety supervisor calls. Today's `fibonacci` is that interface with the robot removed.

The whole picture, one table

	Topic	Service	Action
Shape	stream	question	job
Direction	one way	round trip	round trip + updates
Waiting	nobody waits	caller waits	caller carries on
Cancel	n/a	no	yes
Progress	n/a	no	yes
Typical	/scan, /cmd_vel	reset, query	navigate, follow
Cost	cheapest	cheap	most machinery

Choose the cheapest shape that answers your question. Most robot traffic is topics, and that is correct.

Session B, and what is due

- 1 Build an interfaces package — one custom message, one service.
- 2 Publish and subscribe with your own type.
- 3 Write a service server, call it from the terminal, then from a client.
- 4 **Cause the deadlock on purpose.** Then fix it with `call_async`.
- 5 Add parameters, and change one while the node runs.
- 6 Send a goal to a real action server and watch the feedback.

Assignment 1 is due at the end of this week.

Next week the abstraction ends: `/cmd_vel` reaches a real differential drive, and the robot moves.